

LAYER 2

Data must be diverse —

- web
- books
- code
- mat.
- etc---

↳ but they don't have same weight do
on the {application}

✗ also Choosing tokenizer is important ^{and}

↳ Since each one output different

size, chunk -- etc

so choose good one and Reasonable
one

{deduplication} → for cleaning
generalized

most decoders only LLM uses :

~~X~~ Next token Prediction ~~X~~

→ trillions-of-words games of "Fill in the Blank"

Using CE : $-\sum P(x_{\text{true}}) \log P(x_{\text{pred}})$

~~X~~ Scaling Law :

→ bigger model or better data?

Usually high quality data with
Small model is better

* distributed training

↳ we cant fit GPT-3 in single GPU

↳ thousands of GPU in Parallelism

Data Parallelism

every GPU has a copy but
looks at different sentence

Tensor Parallelism

single math operation
splits across multiple GPU

Pipeline Parallelism

Layer 1 → GPU A }
Layer 2 → GPU B }
etc

ZeRO Sharding



* Fun Fact: in training model weights are usually Smallest Part

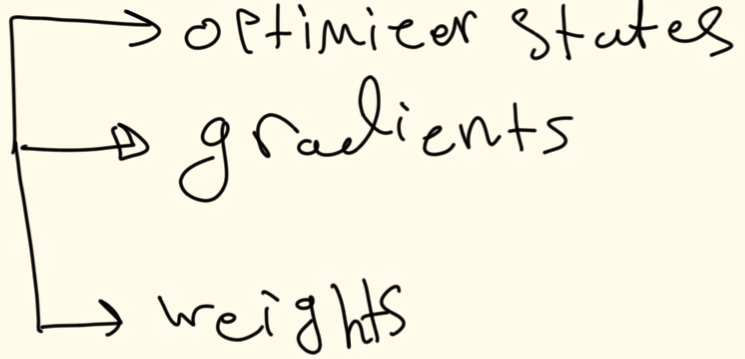
↳ memo hogs: optimizer states and
Gradients are the majority

* in standard Data Parallel

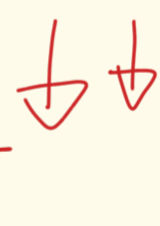
↳ every single GPU keeps full of all ^{copy} three

↳ wasteful → ZeRO sharding

ZeRO sharding: cutting the data into pieces and spreading it across GPUs

slice the $\circ$ 

across GPUs

Speed   memo 

↳ GPUs need to communicate it takes
time